

Proyecto 1: Estrategia de Backup Completo Diario y Verificación Automática

1. Enunciado del ejercicio

Implementar un script en T-SQL que realice un **backup FULL diario** de la base de datos **QhatuPeru**, almacenando el archivo con **fecha en el nombre**.

Después del backup, el sistema debe realizar una verificación de integridad usando **RESTORE VERIFYONLY**.

Finalmente, se debe **crear (mediante script)** un Job de SQL Server Agent que ejecute este proceso automáticamente todos los días.

2. Script de la solución en T-SQL

```
--PROYECTO 1:
DECLARE @BackupPath NVARCHAR(200) = 'C:\SQLBackups\';
DECLARE @FileName NVARCHAR(200);

-- Generar nombre del archivo con fecha
SET @FileName = @BackupPath +
    'QhatuPERU_FULL_' +
    CONVERT(VARCHAR(20), GETDATE(), 112) + '_' +
    REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108), ':', '') +
    '.bak';

-- Crear backup FULL
BACKUP DATABASE QhatuPERU
TO DISK = @FileName
WITH INIT,
    COMPRESSION,
    STATS = 10;

PRINT 'Backup FULL creado: ' + @FileName;

-- Verificar integridad del backup
RESTORE VERIFYONLY
FROM DISK = @FileName;

PRINT 'Verificación de integridad completada correctamente.';
GO
```

```

--CREACIÓN DEL JOB EN SQL SERVER AGENT

USE msdb;
GO

-- 1. Crear el Job
EXEC sp_add_job
    @job_name = N'Backup_Diario_QhatuPERU',
    @enabled = 1,
    @description = N'Realiza backup FULL diario de QhatuPeru y verifica integridad';
GO

-- 2. Crear el paso que ejecutará el backup + verifyonly
EXEC sp_add_jobstep
    @job_name = N'Backup_Diario_QhatuPERU',
    @step_name = N'BackupFULL y Verificación',
    @subsystem = N'TSQL',
    @command = N'
DECLARE @BackupPath NVARCHAR(200) = 'C:\SQLBackups\';
DECLARE @FileName NVARCHAR(200);

SET @FileName = @BackupPath +
    'QhatuPERU_FULL_' +
    CONVERT(VARCHAR(20), GETDATE(), 112) + '_' +
    REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108), ':', '') +
    '.bak';

BACKUP DATABASE QhatuPERU
TO DISK = @FileName
WITH INIT, COMPRESSION, STATS = 10;

RESTORE VERIFYONLY FROM DISK = @FileName;
',
    @retry_attempts = 3,
    @retry_interval = 5;
GO

-- 3. Crear el schedule (diario a las 02:00 AM)
EXEC sp_add_schedule
    @schedule_name = N'Schedule_Backup_Diario_2AM',
    @freq_type = 4, -- Daily
    @freq_interval = 1,
    @active_start_time = 020000; -- 02:00 AM
GO

-- 4. Asociar el schedule al job
EXEC sp_attach_schedule
    @job_name = N'Backup_Diario_QhatuPERU',
    @schedule_name = N'Schedule_Backup_Diario_2AM';
GO

-- 5. Habilitar ejecución con el agente
EXEC sp_add_jobserver
    @job_name = N'Backup_Diario_QhatuPERU',
    @server_name = @@SERVERNAME;
GO

```

3. Justificación técnica de la solución aplicada

Uso de BACKUP DATABASE con compresión

Se utiliza **COMPRESSION** para disminuir el tamaño del archivo, optimizar el espacio y acelerar la transferencia del backup.

Nombre dinámico con fecha y hora

El backup se genera con formato: `QhatuPeru_FULL_YYYYMMDD_HHMMSS.bak`
Esto evita sobrescrituras automáticas y permite almacenar históricos.

RESTORE VERIFYONLY

Se agrega una verificación inmediata para asegurar la integridad del backup antes de guardarlo o transferirlo, detectando: Corrupción en páginas, Estructuras inválidas y Problemas de escritura

Job automatizado en SQL Server Agent

Se implementa el job lo permite que el proceso sea **automático**, repetible y confiable.

4. Explicación de las buenas prácticas utilizadas

I. Separación lógica del código

Se separa:

- Declaración de variables
- Backup
- Verificación
- Creación del job

Esto mejora la mantenibilidad del script.

II. Uso de rutas absolutas y planificadas

El backup se almacena en una carpeta definida (`C:\SQLBackups\`) tal como establecen normas de auditoría.

III. Nombrado estándar de backup

Formato: `Base_TipoBackup_FechaHora.bak`

Estandariza documentación y facilita la recuperación.

IV. Compresión habilitada

Buena práctica recomendada por Microsoft para optimizar almacenamiento.

V. Verificación de integridad

Nunca se debe confiar en un backup sin verificarlo.

VI. Automatización mediante SQL Agent

Evita errores humanos, reduce tiempos y estandariza el proceso.

Proyecto 2: Estrategia Completa — Backup FULL Semanal + Verificación Automática + Auditoría

1. Enunciado del ejercicio

Diseñar e implementar un plan que realice un **backup FULL semanal** de la base de datos **QhatuPERU**, ejecutado **cada domingo a las 02:00 AM**.

Después de generar el backup, el sistema debe **verificar la validez del archivo** mediante **RESTORE VERIFYONLY**.

Finalmente, los resultados del proceso (éxito, error, fecha, nombre del archivo y mensaje del sistema) deben almacenarse en una **tabla de auditoría**.

2. Script de la soluc

```
--PROYECTO 2
-- 1. Creación de tabla de auditoría

USE QhatuPERU;
GO

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'AuditoriaBackup')
BEGIN
    CREATE TABLE AuditoriaBackup (
        IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
        FechaEjecucion DATETIME NOT NULL,
        ArchivoBackup NVARCHAR(300) NOT NULL,
        Resultado NVARCHAR(20) NOT NULL, -- 'OK' o 'ERROR'
        Mensaje NVARCHAR(MAX) NULL
    );
END
GO

-- 2. Script que realizará el backup y registrará auditoría

USE msdb;
GO

DECLARE @BackupPath NVARCHAR(200) = 'C:\SQLBackups\';
DECLARE @FileName NVARCHAR(200);
DECLARE @Mensaje NVARCHAR(MAX) = '';
DECLARE @Resultado NVARCHAR(20) = 'OK';

BEGIN TRY
    -- Generar nombre dinámico del archivo
    SET @FileName = @BackupPath +
        'QhatuPERU_FULL_' +
        CONVERT(VARCHAR(8), GETDATE(), 112) + '_' +
        REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108), ':', '') +
        '.bak';
```

ión en T-SQL

```

-- Crear backup FULL
BACKUP DATABASE QhatuPERU
TO DISK = @FileName
WITH INIT, COMPRESSION, STATS = 10;

-- Verificar integridad
RESTORE VERIFYONLY FROM DISK = @FileName;

SET @Mensaje = 'Backup y verificación completados correctamente.';
END TRY
BEGIN CATCH
    SET @Resultado = 'ERROR';
    SET @Mensaje = ERROR_MESSAGE();
END CATCH;

-- Registrar auditoría
INSERT INTO QhatuPERU.dbo.AuditoriaBackup (FechaEjecucion, ArchivoBackup, Resultado, Mensaje)
VALUES (GETDATE(), @FileName, @Resultado, @Mensaje);

PRINT 'Proceso finalizado. Resultado: ' + @Resultado;
GO

-- 3. Creación del Job que automatiza el proceso

-- Crear Job
EXEC sp_add_job
    @job_name = N'Backup_Semanal_QhatuPERU',
    @enabled = 1,
    @description = N'Job que realiza backup FULL semanal con verificación y auditoría';
GO

```

```

-- Crear Paso del Job
EXEC sp_add_jobstep
    @job_name = N'Backup_Semanal_QhatuPERU',
    @step_name = N'BackupFULL_Verificacion_Auditoria',
    @subsystem = N'TSQL',
    @command = N'
DECLARE @BackupPath NVARCHAR(200) = ''C:\SQLBackups\'';
DECLARE @FileName NVARCHAR(200);
DECLARE @Mensaje NVARCHAR(MAX) = '';
DECLARE @Resultado NVARCHAR(20) = ''OK'';

BEGIN TRY
    SET @FileName = @BackupPath +
        ''QhatuPERU_FULL_'' +
        CONVERT(VARCHAR(8), GETDATE(), 112) + ''_'' +
        REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108), ':', '') +
        ''.bak'';

    BACKUP DATABASE QhatuPERU
    TO DISK = @FileName
    WITH INIT, COMPRESSION, STATS = 10;

    RESTORE VERIFYONLY FROM DISK = @FileName;

    SET @Mensaje = ''Backup y verificación OK.'';
END TRY
BEGIN CATCH
    SET @Resultado = ''ERROR'';
    SET @Mensaje = ERROR_MESSAGE();
END CATCH;

INSERT INTO QhatuPERU.dbo.AuditoriaBackup (FechaEjecucion, ArchivoBackup, Resultado, Mensaje)
VALUES (GETDATE(), @FileName, @Resultado, @Mensaje);
    ',
    @retry_attempts = 3,
    @retry_interval = 5;
GO

```

```

-- Crear schedule SEMANAL (Domingo 02:00 AM)
EXEC sp_add_schedule
    @schedule_name = N'Schedule_Backup_Semanal_Domingo',
    @freq_type = 8,          -- Semanal
    @freq_interval = 1,     -- Domingo
    @active_start_time = 020000;
GO

-- Asociar schedule al Job
EXEC sp_attach_schedule
    @job_name = N'Backup_Semanal_QhatuPERU',
    @schedule_name = N'Schedule_Backup_Semanal_Domingo';
GO

-- Activar job en el servidor actual
EXEC sp_add_jobserver
    @job_name = N'Backup_Semanal_QhatuPERU',
    @server_name = @@SERVERNAME;
GO

```

3. Justificación técnica de la solución aplicada

Backup FULL semanal programado

Se selecciona el domingo a las **02:00 AM** porque:

- Es horario de baja actividad en la mayoría de sistemas productivos.
- Minimiza contención y bloqueos.
- Reduce impacto en el rendimiento.

Uso de TRY...CATCH

Permite capturar errores durante:

- La creación del backup
- La verificación del archivo
- Problemas de acceso a archivos

RESTORE VERIFYONLY

Verifica:

- Estructura interna
- Checksums
- Integridad del backup

Sin necesidad de restaurarlo completamente.

Uso de COMPRESSION

Mejora almacenamiento y rendimiento del backup.

Reintentos automáticos

El job se reintentará **3 veces**, evitando fallos por problemas momentáneos del servidor.

4. Explicación de las buenas prácticas utilizadas

Almacenamiento seguro de Backups

Ruta dedicada `C:\SQLBackups\`
(Estandarización y fácil respaldo externo)

Archivo con fecha y hora

Evita sobrescrituras y mantiene histórico organizado.

Auditoría obligatoria

Toda política de backup debería demostrar:

- Evidencia de ejecución
- Evidencia de fallos
- Evidencia de integridad

Automatización con SQL Agent

Minimiza errores humanos y garantiza cumplimiento regular.

Manejo seguro de errores (TRY/CATCH)

Captura fallos de disco, permisos y corrupción.

Validación del backup antes de registrarlo

Nunca se registra como “OK” si:

- El archivo está corrupto
- No se pudo verificar
- No se generó correctamente

Código organizado y reutilizable

Facilita mantenimiento y auditorías externas.

Proyecto 3: Estrategia Combinada — FULL Semanal + DIFF Diario + LOG Cada 30 Minutos + Restauración Completa

1. Enunciado del ejercicio

Diseñar un plan de respaldo para la base de datos **QhatuPERU** que incluya:

1. **Backup FULL semanal** (se ejecuta una vez por semana).
2. **Backup diferencial diario** (dependiente del último FULL).
3. **Backup del LOG de transacciones cada 30 minutos.**

Además, se debe demostrar cómo realizar la **restauración completa de la cadena**, aplicando en orden:

- Último backup **FULL**
- Último backup **DIFF**
- Todos los backups **LOG** posteriores hasta el momento deseado

2. Script de la solución en T-SQL

2.1 Backup FULL Semanal

```
--Proyecto 3
-- BACKUP FULL SEMANAL - Domingo 02:00 AM

DECLARE @Path NVARCHAR(200) = 'C:\SQLBackups\FULL\';
DECLARE @FileName NVARCHAR(200);

SET @FileName = @Path +
                'QhatuPERU_FULL_' +
                CONVERT(VARCHAR(8), GETDATE(), 112) + '.bak';

BACKUP DATABASE QhatuPERU
TO DISK = @FileName
WITH INIT, COMPRESSION, STATS = 10;
```

2.2 Backup Diferencial Diario

```
--BACKUP DIFERENCIAL DIARIO - 02:00 AM

DECLARE @Path NVARCHAR(200) = 'C:\SQLBackups\DIFF\';
DECLARE @FileName NVARCHAR(200);

SET @FileName = @Path +
                'QhatuPERU_DIFF_' +
                CONVERT(VARCHAR(8), GETDATE(), 112) + '.dif';

BACKUP DATABASE QhatuPERU
TO DISK = @FileName
WITH DIFFERENTIAL,
    INIT,
    COMPRESSION,
    STATS = 10;
```

2.3 Backup LOG cada 30 minutos

```
--BACKUP DEL LOG DE TRANSACCIONES - Cada 30 minutos

DECLARE @Path NVARCHAR(200) = 'C:\SQLBackups\LOG\';
DECLARE @FileName NVARCHAR(200);

SET @FileName = @Path +
                'QhatuPERU_LOG_' +
                CONVERT(VARCHAR(8), GETDATE(), 112) + '_' +
                REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108), ':', '') +
                '.trn';

BACKUP LOG QhatuPERU
TO DISK = @FileName
WITH INIT,
    COMPRESSION,
    STATS = 10;
```

2.4 Programación de los Jobs (demostración con script)

Job 1: FULL semanal — Domingo 02:00 AM

```

--1er Job
EXEC sp_add_job @job_name = 'JOB_FULL_QhatuPERU';

EXEC sp_add_jobstep
    @job_name = 'JOB_FULL_QhatuPERU',
    @step_name = 'BACKUP FULL',
    @subsystem = 'TSQL',
    @command = '<script FULL de arriba>';

EXEC sp_add_schedule
    @schedule_name = 'SCHEDULE_FULL_SEMANAL',
    @freq_type = 8,      -- Semanal
    @freq_interval = 1,  -- Domingo
    @active_start_time = 020000;

EXEC sp_attach_schedule
    @job_name = 'JOB_FULL_QhatuPERU',
    @schedule_name = 'SCHEDULE_FULL_SEMANAL';

EXEC sp_add_jobserver @job_name = 'JOB_FULL_QhatuPERU';

```

Job 2: Diferencial diario — 02:00 AM

```

--2do Job
EXEC sp_add_job @job_name = 'JOB_DIFF_QhatuPERU';

EXEC sp_add_jobstep
    @job_name = 'JOB_DIFF_QhatuPERU',
    @step_name = 'BACKUP DIFERENCIAL',
    @subsystem = 'TSQL',
    @command = '<script DIFF de arriba>';

EXEC sp_add_schedule
    @schedule_name = 'SCHEDULE_DIFF_DIARIO',
    @freq_type = 4,      -- Diario
    @active_start_time = 020000;

EXEC sp_attach_schedule
    @job_name = 'JOB_DIFF_QhatuPERU',
    @schedule_name = 'SCHEDULE_DIFF_DIARIO';

EXEC sp_add_jobserver @job_name = 'JOB_DIFF_QhatuPERU';

```

Job 3: Backup LOG — cada 30 minutos

```

--3er Job
EXEC sp_add_job @job_name = 'JOB_LOG_QhatuPERU';

EXEC sp_add_jobstep
    @job_name = 'JOB_LOG_QhatuPERU',
    @step_name = 'BACKUP LOG',
    @subsystem = 'TSQL',
    @command = '<script LOG de arriba>';

EXEC sp_add_schedule
    @schedule_name = 'SCHEDULE_LOG_30MIN',
    @freq_type = 4,           -- Diario
    @freq_subday_type = 4,   -- Cada X minutos
    @freq_subday_interval = 30,
    @active_start_time = 000000;

EXEC sp_attach_schedule
    @job_name = 'JOB_LOG_QhatuPERU',
    @schedule_name = 'SCHEDULE_LOG_30MIN';

EXEC sp_add_jobserver @job_name = 'JOB_LOG_QhatuPERU';

```

3. Justificación técnica de la solución aplicada

Alta disponibilidad lógica

El backup del LOG permite recuperar la BD hasta el minuto exacto antes de una falla.

Optimización del almacenamiento

DIFF + LOG ocupan mucho menos espacio que realizar FULL diarios.

Jobs completamente automatizados

Reduce riesgo humano y garantiza cumplimiento del plan de respaldo.

Uso de COMPRESSION

Reduce tamaño, tiempos de escritura y uso de disco.

Nombrado estandarizado

Facilita la restauración automática y la gestión del ciclo de vida de archivos.

4. Demostración de Restauración Completa de la Cadena

Supongamos que los archivos son:

- FULL → QhatuPERU_FULL_20250101.bak

- DIFF → QhatuPERU_DIFF_20250104.dif
- LOG
 - QhatuPERU_LOG_20250104_080000.trn
 - QhatuPERU_LOG_20250104_083000.trn
 - QhatuPERU_LOG_20250104_090000.trn

4.1 Restaurar el FULL (con NORECOVERY)

```
--Restaurar el FULL (con NORECOVERY)
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\SQLBackups\FULL\QhatuPERU_FULL_20251120.bak'
WITH NORECOVERY;
```

4.2 Restaurar el último DIFF

```
--Restaurar el último DIFF
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\SQLBackups\DIFF\QhatuPERU_DIFF_20250104.dif'
WITH NORECOVERY;
```

4.3 Aplicar los backups LOG en orden

```
--Aplicar los backups LOG en orden
RESTORE LOG QhatuPERU
FROM DISK = 'C:\SQLBackups\LOG\QhatuPERU_LOG_20250104_080000.trn'
WITH NORECOVERY;

RESTORE LOG QhatuPERU
FROM DISK = 'C:\SQLBackups\LOG\QhatuPERU_LOG_20250104_083000.trn'
WITH NORECOVERY;

RESTORE LOG QhatuPERU
FROM DISK = 'C:\SQLBackups\LOG\QhatuPERU_LOG_20250104_090000.trn'
WITH RECOVERY; -- Este último completa la restauración
```

5. Explicación de las buenas prácticas utilizadas

Estructura de carpetas por tipo de backup (FULL / DIFF / LOG)

Facilita la recuperación y clasificación.

NORECOVERY para mantener la cadena abierta

Indispensable para restaurar DIFF y LOG correctamente.

RECOVERY solo al final

Cierra el proceso y habilita la BD para uso.

Frecuencia de LOG adecuada (cada 30 minutos)

Balance entre pérdida de datos y número de archivos.

Jobs independientes

Aislan responsabilidades:

- Job FULL
- Job DIFF
- Job LOG

Uso de nombres con fecha y hora

Evita sobrescritura y mantiene orden cronológico perfecto.

Proyecto 4: Estrategia Diferencial — Snapshot entre Backups Completos + Retención 14 días

1. Enunciado del ejercicio

Implementar un plan de respaldo diferencial para la base **QhatuPERU** que cumpla:

1. Realizar **backups diferenciales diarios** (de lunes a sábado).
2. Los backups diferenciales deben conservarse por **14 días**.
3. Debe automatizarse un **script de limpieza**, eliminando respaldos antiguos de forma segura.
4. La limpieza debe basarse en la información histórica contenida en `msdb.dbo.backupset`, evitando borrar archivos todavía necesarios.

2. Script de la solución en T-SQL

2.1 Backup Diferencial Diario (Lun–Sáb)

```
--Proyecto 4
--BACKUP DIFERENCIAL (LUNES - SÁBADO)
DECLARE @Path NVARCHAR(200) = 'C:\SQLBackups\DIFF\';
DECLARE @FileName NVARCHAR(300);

SET @FileName = @Path +
                'QhatuPERU_DIFF_' +
                CONVERT(VARCHAR(8), GETDATE(), 112) + '.dif';

BACKUP DATABASE QhatuPERU
TO DISK = @FileName
WITH DIFFERENTIAL,
    INIT,
    COMPRESSION,
    STATS = 10;
```

2.2 Programación del job (solo muestra del script)

```
--Programación del job (solo muestra del script)
EXEC sp_add_job @job_name = 'JOB_DIFERENCIAL_QhatuPERU';

EXEC sp_add_jobstep
    @job_name = 'JOB_DIFERENCIAL_QhatuPERU',
    @step_name = 'BACKUP DIFERENCIAL',
    @subsystem = 'TSQL',
    @command = '<script diferencial de arriba>';

EXEC sp_add_schedule
    @schedule_name = 'SCHEDULE_DIFERENCIAL_LUN_SAB',
    @freq_type = 8,           -- semanal
    @freq_interval = 62,      -- Lunes=2 + Martes=4 + Miércoles=8 + Jueves=16 + Viernes=32
    @active_start_time = 020000; -- 02:00 AM

EXEC sp_attach_schedule
    @job_name = 'JOB_DIFERENCIAL_QhatuPERU',
    @schedule_name = 'SCHEDULE_DIFERENCIAL_LUN_SAB';
```

2.3 Script de limpieza de backups diferenciales (retención de 14 días)

El script elimina SOLO archivos **diferenciales** cuya fecha excede los **14 días**.

Este enfoque es totalmente seguro porque consulta el historial de backups desde `msdb.dbo.backupset`.

```
--LIMPIEZA DE BACKUPS DIFERENCIALES CON RETENCIÓN 14 DÍAS

DECLARE @Ruta NVARCHAR(200) = 'C:\SQLBackups\DIFF\';
DECLARE @Cmd NVARCHAR(500);

;WITH BackupsDiferenciales AS (
    SELECT
        physical_device_name AS Archivo,
        backup_start_date
    FROM msdb.dbo.backupset bs
    INNER JOIN msdb.dbo.backupmediafamily bmf
        ON bs.media_set_id = bmf.media_set_id
    WHERE bs.database_name = 'QhatuPERU'
        AND bs.type = 'I'           -- I = Diferencial
        AND backup_start_date < DATEADD(DAY, -14, GETDATE())
)
SELECT @Cmd = 'DEL "' + Archivo + '"'
FROM BackupsDiferenciales
WHERE Archivo LIKE @Ruta + '%';

-- Ejecutar eliminación
EXEC xp_cmdshell @Cmd, NO_OUTPUT;
```

Notas importantes del script de limpieza

- ✓ Se filtran SOLO backups diferenciales (`type = 'I'`).
- ✓ Se obtiene la ruta exacta desde `backupmediafamily`.

- ✓ Se evita eliminar cualquier archivo reciente (últimos 14 días).
- ✓ Se usa `xp_cmdshell` (requiere estar habilitado).
- ✓ Se puede programar este script como un job diario.

2.4 Job del proceso de limpieza (demostración de script)

```
--Job del proceso de limpieza (demostración de script)
EXEC sp_add_job @job_name = 'JOB_LIMPIEZA_DIFF_QhatuPERU';

EXEC sp_add_jobstep
    @job_name = 'JOB_LIMPIEZA_DIFF_QhatuPERU',
    @step_name = 'LIMPIAR DIFERENCIALES ANTIGUOS',
    @subsystem = 'TSQL',
    @command = '<script de limpieza>';

EXEC sp_add_schedule
    @schedule_name = 'SCHEDULE_LIMPIEZA_01AM',
    @freq_type = 4,          -- Diario
    @active_start_time = 010000;

EXEC sp_attach_schedule
    @job_name = 'JOB_LIMPIEZA_DIFF_QhatuPERU',
    @schedule_name = 'SCHEDULE_LIMPIEZA_01AM';

EXEC sp_add_jobserver @job_name = 'JOB_LIMPIEZA_DIFF_QhatuPERU';
```

3. Justificación técnica de la solución aplicada

Estrategia diferencial como "snapshot" intermedio

Los backups diferenciales actúan como snapshots entre los **FULL semanales**, reduciendo:

- Tiempo de backup diario
- Tamaño de almacenamiento
- Tiempo total de restauración

Conservación durante 14 días

Retención estándar utilizada en ambientes productivos donde hay retención de **2 semanas** (ciclo común en auditorías).

Limpieza basada en backupset

Evita riesgos como:

- Borrar archivos que no correspondan a backups reales

- Borrar archivos necesarios para la restauración
- Error humano al eliminar manualmente archivos

Uso de xp_cmdshell (controlado)

Permite eliminar directamente archivos del sistema con seguridad porque:

- Solo se ejecuta desde un job específico
- Usa información trazable del msdb
- No elimina archivos sin validación previa

Jobs automáticos

Separar el job de respaldo y el job de limpieza mejora:

- Orden del sistema
- Mantenimiento
- Auditoría
- Seguridad

4. Buenas prácticas utilizadas

Respaldos etiquetados con fecha y nombre estándar

Facilita restauraciones y búsquedas.

Carpetas separadas según tipo de backup

FULL / DIFF / LOG

— Mejora orden y reduce errores humanos.

Eliminación automatizada y segura

Basada en la metadata interna del servidor SQL, no en fechas del sistema.

Job específico para limpieza

Evita mezclar lógica de backup con lógica de eliminación.

Retención controlada

14 días cubre respaldos pendientes incluso si falla un FULL semanal.

Uso de COMPRESSION

Menor espacio, mayor velocidad.

PROYECTO 5: Restauración en distintos escenarios — recuperación de corrupción parcial y recuperación de emergencia

1. Enunciado del ejercicio

Practicar restauraciones de la base de datos **QhatuPERU** en tres escenarios diferentes:

- a) **Restauración completa (recuperación estándar):** restaurar un backup FULL funcional y dejar la base operativa.
- b) **Restauración a un punto en el tiempo:** corregir un error humano (ejemplo: eliminación masiva de datos) usando un backup LOG para recuperar justo antes del fallo.
- c) **Restauración ante backup corrupto:** validar integridad, detectar corrupción y restaurar usando un backup alternativo válido.

2. Script de la solución en T-SQL

a) Restauración completa estándar

```
--Proyecto 5
--a) Restauración completa estándar
-- Asegurar que no existan conexiones activas
ALTER DATABASE QhatuPERU SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

-- Restaurar desde backup FULL más reciente
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_FULL_20250120.bak'
WITH REPLACE,
     STATS = 5;

ALTER DATABASE QhatuPERU SET MULTI_USER;
```

b) Restauración punto-en-el-tiempo (Point-in-Time Recovery)

Suposición:

A las 10:25 AM se borraron datos por error.

Se quiere restaurar a las 10:24:59 AM.

```
--b) Restauración punto-en-el-tiempo (Point-in-Time Recovery)

-- Dejar base en modo de un solo usuario
ALTER DATABASE QhatuPERU SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

-- Paso 1: Restaurar FULL en modo NORECOVERY
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_FULL_20250120.bak'
WITH NORECOVERY, REPLACE, STATS = 5;

-- Paso 2: Restaurar último backup de LOG antes del accidente
RESTORE LOG QhatuPERU
FROM DISK = 'C:\Backups\Logs\QhatuPERU_LOG_20250120_1029.trn'
WITH STOPAT = '2025-01-20T10:24:59',
    RECOVERY,
    STATS = 5;

ALTER DATABASE QhatuPERU SET MULTI_USER;
```

c) Restauración ante backup corrupto usando verificación + backup alternativo

```
--c) Restauración ante backup corrupto usando verificación + backup alternativo
-- Verificar integridad del backup
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\QhatuPERU_FULL_20250120.bak';

-- Si aparece un error como:
-- "The backup set is corrupted"
-- entonces se usa un backup alternativo

-- Restauración desde backup alternativo válido
ALTER DATABASE QhatuPERU SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\BackupsSecundarios\QhatuPERU_FULL_20250118.bak'
WITH REPLACE,
    STATS = 5;

ALTER DATABASE QhatuPERU SET MULTI_USER;
```

3. Justificación técnica de la solución aplicada

✓ A— Restauración estándar

Se utiliza un backup FULL como punto más completo y reciente, aplicando **WITH REPLACE** para sobrescribir la base actual en caso de corrupción o fallos.

El modo **SINGLE_USER** evita conflictos con sesiones activas.

✓ B — Recuperación punto-en-el-tiempo

La combinación de:

- FULL (base)
- LOG (detalles del momento exacto del error)
- Parámetro **STOPAT**

permite revertir solo los cambios posteriores al incidente, técnica indispensable ante fallos humanos (DELETE, UPDATE, DROP).

✓C — Backup corrupto

RESTORE VERIFYONLY es la forma formal de detectar integridad de backups.

La solución recae en tener **copias alternas o redundantes**, lo cual imita un escenario real empresarial donde un backup puede dañarse por almacenamiento defectuoso, interrupciones o fallos del motor.

4. Explicación de las buenas prácticas utilizadas

Uso de VERIFYONLY

Antes de restaurar, siempre se valida la integridad para evitar restaurar un archivo dañado.

Manejo de SINGLE_USER / MULTI_USER

Evita bloqueos y asegura restauraciones limpias.

NORECOVERY en restauraciones encadenadas

Permite aplicar múltiples backups (FULL → DIFF → LOG).

STOPAT para restauración precisa

Es fundamental para deshacer errores humanos sin perder todo el día de trabajo.

Copias múltiples y almacenamiento redundante

El proyecto demuestra cómo actuar si un backup está corrupto, algo clave en planes de alta disponibilidad.

Uso de REPLACE

Asegura que la restauración sobrescriba completamente la base actual para evitar conflictos de archivos LDF o MDF.

Rutas organizadas (FULL, DIFF, LOG)

Estandariza la administración y evita confusiones en auditoría.

PROYECTO 6: Backups de LOG para recuperación punto en el tiempo (PITR)

1. Enunciado del ejercicio

Implementar una estrategia de **recuperación punto en el tiempo (PITR)** para la base **QhatuPERU**, cumpliendo:

1. Cambiar el modelo de recuperación a **FULL** (si aún no lo está).
2. Ejecutar **backups de log cada hora** mediante script.
3. Demostrar una **restauración punto en el tiempo**, por ejemplo recuperando la base al momento exacto:
2025-11-15 14:30:00.

2. Script de la solución en T-SQL

a) Cambiar la base al modelo FULL

```
--Proyecto 6
--a) Cambiar la base al modelo FULL
ALTER DATABASE QhatuPERU SET RECOVERY FULL;
GO

-- Recomendado: ejecutar un FULL inmediato al cambiar de modelo
BACKUP DATABASE QhatuPERU
TO DISK = 'C:\Backups\QhatuPERU_FULL_20251115.bak'
WITH INIT, STATS = 5;
GO
```

b) Backup del log cada hora (script demostrativo)

```
--b) Backup del log cada hora (script demostrativo)
-- Simulación de job que se ejecutaría cada hora
BACKUP LOG QhatuPERU
TO DISK = 'C:\Backups\LOGS\QhatuPERU_LOG_20251115_1400.trn'
WITH STATS = 5;

BACKUP LOG QhatuPERU
TO DISK = 'C:\Backups\LOGS\QhatuPERU_LOG_20251115_1500.trn'
WITH STATS = 5;
```

c) Restauración punto en el tiempo: 2025-11-15 14:30:00

Supongamos:

- FULL: QhatuPERU_FULL_20251115.bak
- LOG de 14:00: QhatuPERU_LOG_20251115_1400.trn
- LOG de 15:00: QhatuPERU_LOG_20251115_1500.trn (contiene transacciones hasta después del punto deseado)

Restauración

```
-- C) Restauración
-- Asegurar que nadie use la base
ALTER DATABASE QhatuPERU SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
-- PASO 1: Restaurar el FULL en NORECOVERY
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_FULL_20251115.bak'
WITH NORECOVERY,
    REPLACE,
    STATS = 5;
-- PASO 2: Restaurar backup de LOG de 14:00
RESTORE LOG QhatuPERU
FROM DISK = 'C:\Backups\LOGS\QhatuPERU_LOG_20251115_1400.trn'
WITH NORECOVERY,
    STATS = 5;
-- PASO 3: Restaurar LOG de 15:00 usando STOPAT
RESTORE LOG QhatuPERU
FROM DISK = 'C:\Backups\LOGS\QhatuPERU_LOG_20251115_1500.trn'
WITH STOPAT = '2025-11-15T14:30:00',
    RECOVERY,
    STATS = 5;

ALTER DATABASE QhatuPERU SET MULTI_USER;
```

3. Justificación técnica de la solución aplicada

✓ Modelo FULL

Permite capturar cada cambio realizado en la base mediante backups de LOG, requisito fundamental para PITR.

✓ Backup FULL después del cambio de modelo

El primer log requiere un FULL base para comenzar una nueva cadena coherente.

✓ Backups de log cada hora

Reducen la pérdida máxima de datos a menos de 60 minutos.
En ambientes empresariales incluso pueden ser cada 5 o 15 minutos.

✓ Restauración usando NORECOVERY

Permite aplicar múltiples secuencias de logs sin cerrar la base prematuramente.

✓ STOPAT

Permite recuperar la base exactamente antes de una transacción dañina (fallo humano, borrado accidental, corrupción lógica).

✓ Secuencia de restauración correcta

FULL → LOG 1 → LOG 2 (con STOPAT) → RECOVERY
Es la forma estándar de restaurar cadenas de logs sin perder consistencia.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Modelo FULL para entornos críticos

Evita pérdida significativa de datos y garantiza trazabilidad completa.

Backup FULL tras cambiar de modelo

Obligatorio para iniciar la cadena válida de backups de log.

Automatización en SQL Agent

El backup de log debe ejecutarse vía Job cada hora, no manualmente.

Rutas organizadas

Separar FULL, DIFF y LOG mantiene orden y ayuda en auditorías.

Verificación del orden de restauración

La restauración siempre sigue secuencia cronológica.

Uso de SINGLE_USER

Evita bloqueos durante procesos críticos de restauración.

STOPAT para restauración precisa

Permite regresar segundos antes del error, clave en bancos, ERP, almacenes y comercio electrónico.

PROYECTO 7: Mantenimiento y verificación de copias

1. Enunciado del ejercicio

Implementar un sistema de **verificación periódica** de los backups de la base de datos **QhatuPeru**, que incluya:

1. Un script que ejecute **RESTORE VERIFYONLY** sobre un backup FULL reciente.
2. Registro del resultado (éxito o fallo) en una tabla de auditoría.
3. Generación de una alerta interna: insertar una fila adicional cuando la verificación falle.

Esto simula un sistema real de monitoreo de integridad de backups.

2. Script de la solución en T-SQL

a) Crear tabla de auditoría de verificación

```
--Proyecto 7
--a) Crear tabla de auditoría de verificación
USE QhatuPERU;
GO

CREATE TABLE dbo.AuditoriaVerificacionBackup (
    IdVerificacion INT IDENTITY(1,1) PRIMARY KEY,
    FechaVerificacion DATETIME NOT NULL DEFAULT(GETDATE()),
    RutaBackup VARCHAR(500) NOT NULL,
    Estado VARCHAR(20) NOT NULL,          -- 'OK' o 'ERROR'
    MensajeError VARCHAR(MAX) NULL       -- Información en caso de fallo
);
GO
```

100 %

Messages

Commands completed successfully.

Completion time: 2025-11-20T10:44:13.8838795-05:00

b) Script de verificación con registro y alerta

Este script se puede programar en SQL Agent para correr diariamente.

```
--b) Script de verificación con registro y alerta
DECLARE @BackupPath VARCHAR(500) = 'C:\Backups\QhatuPERU_FULL_ULTIMO.bak';
DECLARE @Resultado VARCHAR(20);
DECLARE @MensajeError VARCHAR(MAX);

BEGIN TRY

    -- Intento de verificación
    RESTORE VERIFYONLY
    FROM DISK = @BackupPath;

    -- Si no hubo error:
    SET @Resultado = 'OK';
    SET @MensajeError = NULL;

END TRY
BEGIN CATCH

    -- Si la verificación falla
    SET @Resultado = 'ERROR';
    SET @MensajeError = ERROR_MESSAGE();

END CATCH;

-- Registrar resultado en la tabla
INSERT INTO dbo.AuditoriaVerificacionBackup (RutaBackup, Estado, MensajeError)
VALUES (@BackupPath, @Resultado, @MensajeError);

-- Generar alerta interna si falló
IF @Resultado = 'ERROR'
BEGIN
    INSERT INTO dbo.AuditoriaVerificacionBackup (RutaBackup, Estado, MensajeError)
    VALUES (
        @BackupPath,
        'ALERTA',
        'La verificación del backup falló. Revisar el archivo inmediatamente.'
    );
END;
```

100 %

Messages

(1 row affected)

(1 row affected)

Completion time: 2025-11-20T10:45:44.7903356-05:00

3. Justificación técnica de la solución aplicada

✓ Verificación periódica de backups

RESTORE VERIFYONLY permite comprobar si el backup es legible y estructuralmente íntegro sin necesidad de restaurarlo completamente.

✓ Uso de TRY-CATCH

Gestiona errores controladamente:

- Si la verificación pasa → registra *OK*
- Si falla → registra *ERROR* y toma acción adicional

✓ Tabla de auditoría

Permite:

- rastrear fallos de forma histórica
- documentar horas, rutas y errores
- facilitar procesos de auditoría y soporte

✓ Alerta mediante registro adicional

Las empresas suelen registrar eventos críticos en tablas específicas para ser leídas por sistemas externos (Power BI, jobs de notificación, correo, etc.).

4. Explicación de las buenas prácticas utilizadas en el proyecto

Auditoría obligatoria en procesos de backup

Siempre se debe contar con un historial de verificaciones de integridad.

TRY-CATCH para control estructurado de errores

Evita que un fallo rompa el proceso automatizado.

Centralización de rutas y parámetros

Uso de variables (@BackupPath) para evitar modificar múltiples líneas al cambiar rutas.

Programación mediante SQL Agent

El script está diseñado para ejecutarse diariamente sin intervención manual.

Señalización de errores mediante filas adicionales

Esta técnica permite integrar notificaciones automáticas hacia:

- correos de administrador
- paneles de monitoreo
- alertas de soporte

PROYECTO 8: Restauraciones en distintos escenarios (corrupción parcial, hardware, pérdida del log)

1. Enunciado del ejercicio

Diseñar y ejecutar procedimientos de restauración para tres escenarios diferentes:

A) Corrupción parcial en el archivo de datos (MDF) en el mismo servidor.

Se debe restaurar desde un backup válido y dejar la base operativa.

B) Pérdida total del archivo de log (LDF).

Se debe restaurar usando backups FULL y diferenciales (si existen), ya que el log no puede recuperarse.

C) Restauración en un servidor diferente (migración).

Simular mover la base QhatuPERU a otro servidor mediante copia y restauración de backups.

2. Script de la solución en T-SQLA) Escenario A — Archivo MDF corrupto (misma máquina)

Supuesto: QhatuPERU.mdf está dañado.

Pasos:

```
--Proyecto 8
-- Forzar modo single user para evitar bloqueos
ALTER DATABASE QhatuPERU SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

-- Restauración del backup FULL más reciente
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_FULL_20251118.bak'
WITH REPLACE,
    STATS = 5;

ALTER DATABASE QhatuPERU SET MULTI_USER;
```

Si existe un backup diferencial reciente, se inserta así:

```
--Si existe un backup diferencial reciente
-- Después del FULL, aplicar el DIFF
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_DIFF_20251118.bak'
WITH RECOVERY,
    STATS = 5;
```

B) Escenario B — Pérdida del archivo de log (LDF)

Situación:

El archivo `QhatuPERU_log.ldf` se perdió o dañó.

En este caso **NO** se puede usar backups de LOG; se debe restaurar usando FULL + DIFF.

Pasos:

```
--b
-- Poner base en modo emergencia si está inaccesible
ALTER DATABASE QhatuPERU SET EMERGENCY;
GO

-- Establecer modo single user
ALTER DATABASE QhatuPERU SET SINGLE_USER;

-- Restauración completa con REPLACE (recrea nuevo LDF)
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_FULL_20251115.bak'
WITH NORECOVERY,
    REPLACE,
    STATS = 5;

-- Aplicar diferencial si existe
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_DIFF_20251115.bak'
WITH RECOVERY,
    STATS = 5;

ALTER DATABASE QhatuPERU SET MULTI_USER;
```

C) Escenario C — Restauración en otro servidor (migración)

Supuesto:

Deseas mover QhatuPERU del **Servidor A** al **Servidor B**.

Pasos en SERVIDOR A: hacer el backup

```
--c
--SERVIDOR A: hacer el backup
BACKUP DATABASE QhatuPERU
TO DISK = 'D:\Migracion\QhatuPERU_MIGRACION_FULL_20251120.bak'
WITH INIT, STATS = 5;
```

Copiar el archivo .bak al otro servidor.

Pasos en SERVIDOR B: restaurar

```
--SERVIDOR B: restaurar
-- Crear carpeta destino si no existe
-- Restaurar la base indicando nuevas rutas para MDF y LDF

RESTORE DATABASE QhatuPERU
FROM DISK = 'D:\Migracion\QhatuPERU_MIGRACION_FULL_20251120.bak'
WITH MOVE 'QhatuPERU'
    TO 'D:\SQLData\QhatuPERU.mdf',
    MOVE 'QhatuPERU_log'
    TO 'D:\SQLLogs\QhatuPERU_log.ldf',
    REPLACE,
    STATS = 5;
```

3. Justificación técnica de la solución aplicada

✓ Escenario A: Archivo MDF corrupto

- La corrupción del MDF hace imposible reparar la base sin pérdida.
- La solución correcta es **restaurar desde un backup válido**.
- **REPLACE** asegura que la base dañada sea sobrescrita.

✓ Escenario B: Pérdida del archivo LDF

- Sin el archivo LDF, no se puede hacer ninguna recuperación de logs.
- Por eso se usa la secuencia: FULL → DIFF.
- SQL Server genera un nuevo LDF usando **WITH REPLACE**.

✓ Escenario C: Migración a otro servidor

- Se realiza siempre con backups, nunca copiando MDF/LDF directamente.
- El uso de **WITH MOVE** permite adaptar rutas diferentes en el servidor destino.
- Es el método más seguro y recomendado por Microsoft.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Uso de SINGLE_USER

Evita sesiones que puedan bloquear la restauración.

Manejo correcto de NORECOVERY y RECOVERY

Permite aplicar múltiples capas de backups sin cerrar la base de manera prematura.

Copias de seguridad como medio oficial de migración

Nunca se deben copiar MDF/LDF manualmente.

Diferenciar FULL, DIFF y LOG según cada escenario

- Corrupción: FULL + DIFF
- Pérdida de LOG: FULL + DIFF (NO se usan logs)
- Migración: FULL (o FULL + DIFF si es necesario)

Rutas nuevas mediante WITH MOVE

Evita errores cuando el servidor destino tiene carpetas diferentes.

REPLACE

Es indispensable cuando el archivo MDF corrupto o faltante no puede usarse.

PROYECTO 9: Uso práctico de BACKUP DATABASE y RESTORE DATABASE con MOVE (cambio de archivos físicos)

1. Enunciado del ejercicio

Simular un escenario donde la base de datos **QhatuPERU** debe mover sus archivos físicos (.mdf y .ldf) a otra unidad del servidor debido a falta de espacio en el disco actual.

Objetivo:

1. Generar un **backup completo** de la base.
2. Restaurarla en otra ubicación usando:
`RESTORE DATABASE ... WITH MOVE`
3. Demostrar cómo el MDF y LDF se reubican en nuevas rutas (por ejemplo, de `C:\` a `D:\`).

2. Script de la solución en T-SQL

a) Ver rutas actuales de los archivos

```
--Proyecto 9
--a) Ver rutas actuales de los archivos (opcional, pero recomendado)
USE QhatuPERU;
GO

SELECT
    name AS Archivo,
    physical_name AS RutaActual,
    type_desc AS Tipo
FROM sys.database_files;
```

20 %

Results Messages

	Archivo	RutaActual	Tipo
1	QhatuPERU_Primary	C:\SQLData\QhatuPERU_Primary.mdf	ROWS
2	QhatuPeru_Log	C:\SQLLogs\QhatuPERU_Log.ldf	LOG
3	QhatuPeru_Secun...	C:\SQLData\QhatuPERU_Secundar...	ROWS

Ejemplo de salida típica:

- MDF: `C:\SQLData\QhatuPERU.mdf`
- LDF: `C:\SQLLogs\QhatuPERU_log.ldf`

b) Crear el backup FULL

```
--b) Crear el backup FULL
BACKUP DATABASE QhatuPERU
TO DISK = 'C:\Backups\QhatuPERU_FULL_MOVER_20251120.bak'
WITH INIT, STATS = 10;
GO
```

c) Poner la base en modo SINGLE_USER y desmontar

```
--c) Poner la base en modo SINGLE_USER y desmontar
ALTER DATABASE QhatuPERU SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
```

100 %

Messages

Commands completed successfully.

Completion time: 2025-11-20T14:48:29.9009754-05:00

d) Restaurar en la NUEVA ubicación usando WITH MOVE

En este ejemplo moveremos la base a unidades nuevas:

- D:\SQLData\ para el MDF
- D:\SQLLogs\ para el LDF

```
--d) Restaurar en la NUEVA ubicación usando WITH MOVE
RESTORE DATABASE QhatuPERU
FROM DISK = 'C:\Backups\QhatuPERU_FULL_MOVER_20251120.bak'
WITH
    MOVE 'QhatuPERU' TO 'C:\SQLData\QhatuPERU.mdf',
    MOVE 'QhatuPERU_log' TO 'C:\SQLLogs\QhatuPERU_log.ldf',
    REPLACE,
    RECOVERY,
    STATS = 10;
GO
```

e) Devolver la base a MULTI_USER

```
--e) Devolver la base a MULTI_USER
ALTER DATABASE QhatuPERU SET MULTI_USER;
GO
```

00 %

Messages

Commands completed successfully.

Completion time: 2025-11-20T14:50:39.0679856-05:00

f) Confirmar la nueva ruta

```
--f) Confirmar la nueva ruta
SELECT
    name AS Archivo,
    physical_name AS NuevaRuta,
    type_desc AS Tipo
FROM sys.database_files;
```

100 %

Results Messages

	Archivo	NuevaRuta	Tipo
1	QhatuPERU_Primary	C:\SQLData\QhatuPERU_Primary.mdf	ROWS
2	QhatuPeru_Log	C:\SQLLogs\QhatuPERU_Log.ldf	LOG
3	QhatuPeru_Secundario	C:\SQLData\QhatuPERU_Secundario.ndf	ROWS

3. Justificación técnica de la solución aplicada

✓ Backup previo obligatorio

Mover archivos directamente en el filesystem *corrompería* la base.
Por eso se usa **BACKUP** → **RESTORE**, que es el método oficial y seguro.

✓ Uso de WITH MOVE

Permite indicar a SQL Server una nueva ruta física para los archivos al restaurar.
Es indispensable cuando la base **ya existe** pero debe moverse.

✓ WITH REPLACE

Sobrescribe la base existente en caso de conflicto.
Esto es necesario cuando se mueve dentro del mismo servidor.

✓ Modo SINGLE_USER

Asegura que nadie mantenga conexiones activas que bloqueen la restauración.

✓ Verificación final de rutas

Comprobar con `sys.database_files` valida que el movimiento fue exitoso.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Nunca mover MDF/LDF manualmente

Es una mala práctica y puede dejar la base inutilizable.
Siempre se debe usar una operación RESTORE con MOVE.

Mantener carpetas separadas para datos y logs

Ayuda al rendimiento y facilita futuras migraciones.

Etiquetar los backups con nombre y fecha

Facilita auditoría y evita confusiones entre archivos.

Usar REPLACE solo cuando sea seguro

Evita sobreescrituras accidentales.
Aquí es correcto porque estamos migrando a otro disco.

Validar rutas antes y después

Comprobar ubicaciones previas y nuevas garantiza trazabilidad en el proyecto.

Planificación para escenarios reales

Mover archivos es una tarea común cuando:

- se agota el espacio
- se muda a discos más rápidos
- se reorganiza almacenamiento empresarial

PROYECTO 10: Verificación y mantenimiento de backups — Automatización y chequeos completos

1. Enunciado del ejercicio

Diseñar un proceso automatizado que:

a) Verifique todos los archivos .bak dentro de una carpeta de backups usando **RESTORE VERIFYONLY**.

b) Compare los resultados con la información histórica contenida en **msdb.dbo.backupset** para validar que cada backup tenga un registro esperado.

c) Inserte un resumen de todos los resultados en una tabla **BackupAudit**, simulando el reporte del job.

2. Script de la solución en T-SQL

a) Crear tabla de auditoría (BackupAudit)

```
--Proyecto 10
--a) Crear tabla de auditoría (BackupAudit)
USE QhatuPERU;
GO

CREATE TABLE dbo.BackupAudit (
    IdAudit INT IDENTITY(1,1) PRIMARY KEY,
    FechaEjecucion DATETIME NOT NULL DEFAULT(GETDATE()),
    NombreBackup VARCHAR(300),
    RutaBackup VARCHAR(500),
    Estado VARCHAR(20),           -- OK / ERROR
    Mensaje VARCHAR(MAX) NULL,
    ExisteEnMSDB BIT              -- 1 = sí, 0 = no
);
GO
```

b) Script del job automatizado (verifica todos los .bak)

Este script se ejecutaría desde un job de SQL Agent, pero aquí se muestra como un bloque ejecutable.

```

--b) Script del job automatizado (verifica todos los .bak)
DECLARE @Carpeta VARCHAR(500) = 'C:\Backups\';
DECLARE @cmd VARCHAR(1000);

-- Tabla temporal para almacenar los archivos encontrados
CREATE TABLE #ArchivosBackup (
    NombreArchivo VARCHAR(300),
    RutaCompleta VARCHAR(500)
);

-- Obtener lista de archivos .bak desde la carpeta
SET @cmd = 'dir "' + @Carpeta + '*.bak' /b';
INSERT INTO #ArchivosBackup(NombreArchivo)
EXEC master.sys.xp_cmdshell @cmd;

-- Eliminar null o filas no válidas
DELETE FROM #ArchivosBackup WHERE NombreArchivo IS NULL;

-- Concatenar ruta completa
UPDATE #ArchivosBackup
SET RutaCompleta = @Carpeta + NombreArchivo;

-- Variables de trabajo
DECLARE @FileName VARCHAR(500),
        @Estado VARCHAR(20),
        @Mensaje VARCHAR(MAX),
        @ExisteMSDB BIT;

DECLARE cursorFiles CURSOR FOR
    SELECT RutaCompleta FROM #ArchivosBackup;

OPEN cursorFiles;
FETCH NEXT FROM cursorFiles INTO @FileName;

WHILE @@FETCH_STATUS = 0
BEGIN
    BEGIN TRY
        -----
        -- 1. Verificar el backup
        -----
        RESTORE VERIFYONLY FROM DISK = @FileName;
        SET @Estado = 'OK';
        SET @Mensaje = NULL;
    END TRY
    BEGIN CATCH
        SET @Estado = 'ERROR';
        SET @Mensaje = ERROR_MESSAGE();
    END CATCH;

```

```

-- 2. Validar si existe en MSDB
-----
SET @ExisteMSDB = 0;

IF EXISTS (
    SELECT 1
    FROM msdb.dbo.backupset
    WHERE physical_device_name LIKE '%' + @FileName + '%'
)
    SET @ExisteMSDB = 1;

-----
-- 3. Insertar resumen en BackupAudit
-----
INSERT INTO dbo.BackupAudit (NombreBackup, RutaBackup, Estado, Mensaje, ExisteEnMSDB)
VALUES (
    PARSENAME(REPLACE(@FileName, '.', '_'), 2),
    @FileName,
    @Estado,
    @Mensaje,
    @ExisteMSDB
);

FETCH NEXT FROM cursorFiles INTO @FileName;
END

    FETCH NEXT FROM cursorFiles INTO @FileName;
END

CLOSE cursorFiles;
DEALLOCATE cursorFiles;

DROP TABLE #ArchivosBackup;

```

3. Justificación técnica de la solución aplicada

✓ Uso de xp_cmdshell

Permite listar archivos del sistema operativo para detectar todos los **.bak** disponibles.
Es la única forma de automatizar la verificación de todos los archivos en una carpeta.

✓ RESTORE VERIFYONLY

Es una verificación estructural del backup sin restaurarlo.
Permite saber si:

- el archivo está completo
- no está corrupto
- la cabecera es válida

✓ Comparación con msdb.backupset

Garantiza coherencia entre:

- archivos presentes en disco
- historial de backups que SQL Server registró

Esto detecta:

- archivos abandonados
- archivos sin registro
- inconsistencias entre backupset y el directorio físico

✓ **Tabla BackupAudit**

Permite consolidar:

- estado de cada archivo
- errores detectados
- existencia en MSDB
- información para auditoría técnica

Es el equivalente a un “log” del job.

✓ **Uso de cursor**

Es apropiado aquí porque se requiere procesar **archivo por archivo** de forma secuencial.

4. Explicación de las buenas prácticas utilizadas

Auditoría obligatoria

Toda operación crítica debe registrarse para análisis posterior.

Automatización con un job SQL Agent

Evita errores manuales y asegura ejecución diaria/semanal.

Manejo seguro de excepciones (TRY-CATCH)

Permite continuar el proceso aun cuando un respaldo esté dañado.

Separación clara de responsabilidades

- Detección de archivos
- Verificación de integridad
- Validación contra MSDB
- Registro en auditoría

Evitar restaurar backups innecesariamente

VERIFYONLY evita costos y riesgos de una restauración completa.

Limpieza y control de resultados

La tabla **BackupAudit** sirve como centro de monitoreo.

Proyecto 11: Plan de mantenimiento — Limpieza y retención usando msdb

1. Enunciado del ejercicio

Crear un plan de mantenimiento básico completamente automatizado mediante scripts T-SQL que realice lo siguiente:

1. **Eliminar historiales de backup antiguos** usando `msdb.dbo.sp_delete_backuphistory`.
2. **Mantener un registro organizado de los backups realizados**, consultando `msdb.dbo.backupset`.
3. **Eliminar archivos físicos .BAK y .TRN** con más de *X días* (por ejemplo, 15 días), desde el sistema de archivos mediante `xp_cmdshell` o **PowerShell** ejecutado desde SQL Server Agent.

El objetivo es reducir la saturación del msdb y limpiar archivos viejos para ahorrar espacio.

2. Script de la solución en T-SQL

IMPORTANTE: Cambia las rutas según tu entorno.

2.1. Configurar el número de días de retención

```
--Proyecto 11
--2.1. Configurar el número de días de retención
DECLARE @DiasRetencion INT = 15;
```

2.2. Limpieza de historiales antiguos

```
--2.2. Limpieza de historiales antiguos
DECLARE @FechaLimite DATETIME;
SET @FechaLimite = DATEADD(DAY, -@DiasRetencion, GETDATE());

EXEC msdb.dbo.sp_delete_backuphistory @FechaLimite;
```

2.3. Limpieza de archivos .BAK y .TRN antiguos en carpeta

Se usa `xp_cmdshell`. Debes activarla si no está habilitada.

Habilitar xp_cmdshell (si es necesario)

```
--2.3. Limpieza de archivos .BAK y .TRN antiguos en carpeta
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE;
```

00 %

Messages

Configuration option 'show advanced options' changed from 1 to 1. Run the RECONFIGURE statement to install.
Configuration option 'xp_cmdshell' changed from 0 to 1. Run the RECONFIGURE statement to install.

Completion time: 2025-11-20T15:02:56.7989896-05:00

2.4. Eliminar archivos .BAK y .TRN viejos (PowerShell desde SQL)

Ruta de ejemplo: `C:\Backups\QhatuPeru\`

```
--2.4. Eliminar archivos .BAK y .TRN viejos (PowerShell desde SQL)
DECLARE @BackupPath NVARCHAR(255) = 'C:\Backups\QhatuPeru\';
DECLARE @PS NVARCHAR(MAX);

SET @PS = '
$Limit = (Get-Date).AddDays(-' + CAST(@DiasRetencion AS NVARCHAR(10)) + ');
Get-ChildItem "' + @BackupPath + '" -Filter *.bak | Where-Object { $_.LastWriteTime -lt $Limit } | Remove-Item -Force;
Get-ChildItem "' + @BackupPath + '" -Filter *.trn | Where-Object { $_.LastWriteTime -lt $Limit } | Remove-Item -Force;
';

EXEC xp_cmdshell 'powershell -Command "& { ' + @PS + ' }";'
```

2.5. Registro de resumen en tabla de auditoría

```
--2.5. Registro de resumen en tabla de auditoría
USE QhatuPeru;
GO

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'BackupMaintenanceAudit')
BEGIN
    CREATE TABLE BackupMaintenanceAudit (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        FechaEjecucion DATETIME DEFAULT GETDATE(),
        DiasRetencion INT,
        CarpetasProcesadas VARCHAR(500),
        Estado VARCHAR(20),
        Detalle VARCHAR(MAX)
    );
END

INSERT INTO BackupMaintenanceAudit (DiasRetencion, CarpetasProcesadas, Estado, Detalle)
VALUES (@DiasRetencion, @BackupPath, 'OK', 'Limpieza de historiales y archivos antiguos completada');
```

2.6. Crear un JOB automatizado (demostración por script)

```
--2.6. Crear un JOB automatizado (demostración por script)
USE msdb;
GO

EXEC sp_add_job
    @job_name = 'JOB_Mantenimiento_Backups_QhatuPeru';

EXEC sp_add_jobstep
    @job_name = 'JOB_Mantenimiento_Backups_QhatuPeru',
    @step_name = 'Limpieza_Backups_Antiguos',
    @subsystem = 'TSQL',
    @command = 'EXEC QhatuPeru.dbo.usp_MantenimientoBackups;',
    @retry_attempts = 3,
    @retry_interval = 5;

EXEC sp_add_schedule
    @schedule_name = 'Schedule_Diario_03AM',
    @freq_type = 4, -- diario
    @freq_interval = 1,
    @active_start_time = 030000; -- 3:00 AM

EXEC sp_attach_schedule
    @job_name = 'JOB_Mantenimiento_Backups_QhatuPeru',
    @schedule_name = 'Schedule_Diario_03AM';

EXEC sp_add_jobserver
    @job_name = 'JOB_Mantenimiento_Backups_QhatuPeru';
```

Nota: El script principal debe estar dentro de un procedimiento almacenado llamado `usp_MantenimientoBackups`.

3. Justificación técnica de la solución aplicada

1. `sp_delete_backuphistory`

Evita que la base `msdb` crezca demasiado, lo que afecta el rendimiento del SQL

Server Agent y las herramientas de administración.

2. Eliminación de archivos físicos

Los archivos .BAK y .TRN ocupan espacio significativo.

Se usa PowerShell porque permite un control detallado de fechas y filtros sin complejidad.

3. Uso de xp_cmdshell

Es la opción más directa para ejecutar scripts PowerShell desde SQL Server.

Solo se habilita cuando es necesario.

4. Auditoría centralizada

La tabla BackupMaintenanceAudit permite verificar fecha, estado y detalles de cada ejecución.

Esto facilita auditorías futuras y cumplimiento normativo.

5. Job SQL Agent

Permite automatizar el mantenimiento, manteniendo limpio el entorno y asegurando continuidad del plan sin intervención manual.

4. Explicación de las buenas prácticas utilizadas

✓ Separación de lógica:

Todo el flujo se encapsula en un procedimiento almacenado, facilitando mantenimiento y cambios futuros.

✓ Automatización total:

Evita errores humanos y garantiza que el mantenimiento ocurra incluso si el DBA no está disponible.

✓ Retención configurable:

Permite ajustar el número de días sin modificar el código principal.

✓ Uso controlado de xp_cmdshell:

Se habilita solo si es necesario y se recomienda deshabilitarla después.

✓ Registro de auditoría:

Proporciona trazabilidad y un historial verificable.

✓ Uso de PowerShell:

Evita ciclos complejos en T-SQL y permite manipular archivos de forma robusta.

✓ Ejecución fuera de horas pico (3 AM):

Minimiza impacto en el rendimiento del servidor.

Proyecto 12: Uso avanzado de BACKUP DATABASE y RESTORE DATABASE con opciones

1. Enunciado del ejercicio

El objetivo es practicar y demostrar el uso de las opciones avanzadas de **BACKUP DATABASE** y **RESTORE DATABASE**, incluyendo:

- `COPY_ONLY`
- `FORMAT`
- `WITH CHECKSUM`
- `WITH RETAINDAYS`
- `WITH STATS`
- `WITH MOVE`
- `WITH REPLACE`

Estas opciones permiten controlar backups que no afecten la cadena de diferenciales, establecer retención, verificar integridad, crear conjuntos nuevos, mover archivos durante restauración y sobrescribir bases cuando es necesario.

El estudiante debe ejecutar scripts que utilicen todas estas opciones y demostrar restauraciones con manejo avanzado de archivos.

2. Script de la solución (T-SQL)

Cambia las rutas según tu entorno.

2.1. BACKUP con COPY_ONLY (no altera cadena de diferenciales)

```
-- Proyecto 12
-- 2.1. BACKUP con COPY_ONLY (no altera cadena de diferenciales)
BACKUP DATABASE QhatuPeru
TO DISK = 'C:\Backups\QhatuPeru_COPY_ONLY.bak'
WITH COPY_ONLY,
CHECKSUM,
STATS = 10;
```

2.2. BACKUP con FORMAT (nuevo conjunto, borra contenido anterior del media)

```

--2.2. BACKUP con FORMAT (nuevo conjunto, borra contenido anterior del media)
BACKUP DATABASE QhatuPeru
TO DISK = 'C:\Backups\QhatuPeru_FORMAT.bak'
WITH FORMAT, -- Borra todo lo previo en el archivo .bak
INIT, -- Inicializa el archivo
CHECKSUM,
STATS = 5;

```

2.3. BACKUP con RETAIN_DAYS (protege backup de sobrescritura por X días)

```

--2.3. BACKUP con RETAIN_DAYS (protege backup de sobrescritura por X días)
BACKUP DATABASE QhatuPeru
TO DISK = 'C:\Backups\QhatuPeru_Retain7.bak'
WITH RETAIN_DAYS = 7, -- No puede ser sobrescrito por 7 días
CHECKSUM,
STATS = 10;

```

2.4. BACKUP LOG con CHECKSUM y STATS

```

--2.4. BACKUP LOG con CHECKSUM y STATS
BACKUP LOG QhatuPeru
TO DISK = 'C:\Backups\Log_QhatuPeru.trn'
WITH CHECKSUM,
STATS = 10;

```

2.5. RESTORE VERIFYONLY (validar backup con checksum)

```

--2.5. RESTORE VERIFYONLY (validar backup con checksum)
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\QhatuPeru_FORMAT.bak'
WITH CHECKSUM;

```

2.6. Restauración con MOVE (mover archivos físicos a otra ruta)

Simula que movemos la base a otra unidad o carpeta.

```

--2.6. Restauración con MOVE (mover archivos físicos a otra ruta)
RESTORE DATABASE QhatuPeru_TEST
FROM DISK = 'C:\Backups\QhatuPeru_FORMAT.bak'
WITH MOVE 'QhatuPeru' TO 'C:\Data\QhatuPeru_TEST.mdf',
MOVE 'QhatuPeru_log' TO 'C:\Logs\QhatuPeru_TEST_log.ldf',
REPLACE, -- Sobrescribe si existe
RECOVERY,
STATS = 5;

```

2.7. Restauración con REPLACE (forzar sobrescritura)

```
--2.7. Restauración con REPLACE (forzar sobrescritura)
RESTORE DATABASE QhatuPeru
FROM DISK = 'C:\Backups\QhatuPeru_Retain7.bak'
WITH REPLACE,
RECOVERY,
STATS = 10;
```

2.8. Restauración NO-RECOVERY (preparar cadena)

```
--2.8. Restauración NO-RECOVERY (preparar cadena)
RESTORE DATABASE QhatuPeru_TEST
FROM DISK = 'C:\Backups\QhatuPeru_FORMAT.bak'
WITH NORECOVERY;
```

2.9. Restauración de LOG posterior

```
--2.9. Restauración de LOG posterior
RESTORE LOG QhatuPeru_TEST
FROM DISK = 'C:\Backups\Log_QhatuPeru.trn'
WITH RECOVERY,
STATS = 5;
```

3. Justificación técnica de la solución aplicada

1. COPY_ONLY

Se usa cuando se desea un backup sin alterar la cadena de backups diferenciales. Esto evita problemas con trabajos de backup tradicionales.

2. FORMAT + INIT

Permite iniciar un nuevo conjunto de backup en un archivo, eliminando cualquier backup previo almacenado allí. Es útil en ciclos de mantenimiento.

3. CHECKSUM

Garantiza que los datos leídos y escritos sean coherentes. Reduce la posibilidad de restaurar un backup corrupto.

4. RETAINDAYS

Previene sobrescritura accidental del archivo por cierto tiempo, actuando como medida de protección.

5. STATS

Muestra el progreso de la operación, útil para bases grandes.

6. MOVE

Se usa al restaurar para cambiar la ubicación física de los archivos .mdf y .ldf. Importante en migraciones, nuevas unidades o reorganización de almacenamiento.

7. **REPLACE**

Forza la restauración sobre una base existente sin verificar si los backups coinciden.

Se usa con extremo cuidado, por ejemplo, en ambientes de desarrollo o restauraciones aseguradas.

8. **NORECOVERY**

Deja la base en estado pendiente para permitir aplicar múltiples logs en secuencia.

4. Explicación de las buenas prácticas

✓ **Uso de CHECKSUM siempre que sea posible**

Evita restauraciones corruptas.

✓ **COPY_ONLY antes de tareas críticas**

Permite tomar backups manuales sin romper la cadena de otros planes automáticos.

✓ **STATS en bases grandes**

Mejora el monitoreo y la administración en tiempo real.

✓ **RETAIN_DAYS como medida de seguridad**

Evita sobrescribir backups recientes por error.

✓ **Separación de rutas (Data / Logs)**

Distribuye I/O y mejora rendimiento.

✓ **MOVE durante restauración**

Es fundamental en ambientes donde los nombres lógicos no coinciden con rutas físicas.

✓ **REPLACE solo cuando se sabe lo que se hace**

Evita pérdida accidental de bases de producción.

✓ **Usar NORECOVERY al restaurar cadenas completas**

Es la forma correcta de restaurar FULL + diffs + logs.

Proyecto 13: Restauración punto en el tiempo (recuperar una fila borrada accidentalmente)

1. Enunciado del ejercicio

Simular un escenario donde se eliminan filas por error en la tabla **ARTICULO** de la base de datos **QhatuPeru**.

El objetivo es recuperar los datos usando una **restauración punto-en-el-tiempo (PITR)**:

1. Realizar **FULL backup** inicial.
2. Realizar **LOG backups** para permitir recuperación a un instante exacto.
3. Simular el borrado accidental (**DELETE**).
4. Restaurar la base **hasta segundos antes del error** en una base temporal llamada **QhatuPeru_Rescate**.
5. Extraer las filas perdidas y reinsertarlas en la base original.

2. Script de la solución en T-SQL

Ajusta rutas y fechas según tu entorno.

Este proceso **no interrumpe** la base original. La restauración se hace en una **base paralela**.

2.1. Preparación: asegurarse que el modelo de recuperación sea FULL

```
--Proyecto 13
--2.1. Preparación: asegurarse que el modelo de recuperación sea FULL
ALTER DATABASE QhatuPeru SET RECOVERY FULL;
GO
```

2.2. Tomar un BACKUP FULL inicial

```
--2.2. Tomar un BACKUP FULL inicial
BACKUP DATABASE QhatuPeru
TO DISK = 'C:\Backups\QhatuPeru_FULL.bak'
WITH INIT, CHECKSUM, STATS = 10;
GO
```

2.3. Realizar un BACKUP DE LOG antes de la operación riesgosa

```
--2.3. Realizar un BACKUP DE LOG antes de la operación riesgosa
BACKUP LOG QhatuPeru
TO DISK = 'C:\Backups\QhatuPeru_Log_1.trn'
WITH INIT, CHECKSUM, STATS = 10;
GO
```

2.4. Simulación del borrado accidental

```
--2.4. Simulación del borrado accidental
-- Antes: revisar la fila que se eliminará
SELECT * FROM ARTICULO WHERE IdArticulo = 5;

-- Error humano simulando eliminación accidental
DELETE FROM ARTICULO WHERE IdArticulo = 5;
GO
```

2.5. Realizar LOG backup inmediatamente después del error

(se requiere para delimitar la cadena)

```
--2.5. Realizar LOG backup inmediatamente después del error

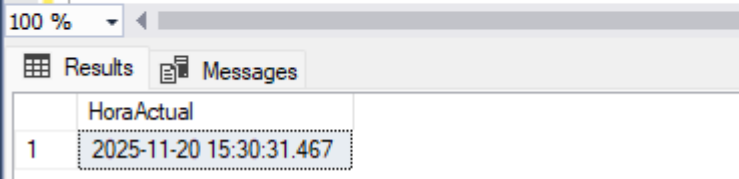
BACKUP LOG QhatuPeru
TO DISK = 'C:\Backups\QhatuPeru_Log_2_afterError.trn'
WITH CHECKSUM, STATS = 10;
GO
```

2.6. Determinar hora exacta del error

Ejemplo aproximado (ajústalo en tu caso real):

```
--2.6. Determinar hora exacta del error

SELECT GETDATE() AS HoraActual;
```



	HoraActual
1	2025-11-20 15:30:31.467

Suponemos que el DELETE ocurrió a:

2025-11-15 14:35:20

Queremos restaurar la base al instante:

2025-11-15 14:35:15 (cinco segundos antes del error)

2.7. Restaurar la base en una base temporal (PITR)

```

--2.7. Restaurar la base en una base temporal (PITR)
-- 1. Restaurar FULL en modo NORECOVERY
RESTORE DATABASE QhatuPeru_Rescate
FROM DISK = 'C:\Backups\QhatuPeru_FULL.bak'
WITH MOVE 'QhatuPeru' TO 'D:\Data\QhatuPeru_Rescate.mdf',
      MOVE 'QhatuPeru_log' TO 'D:\Logs\QhatuPeru_Rescate_log.ldf',
      NORECOVERY;

-- 2. Aplicar LOG previo al error
RESTORE LOG QhatuPeru_Rescate
FROM DISK = 'C:\Backups\QhatuPeru_Log_1.trn'
WITH NORECOVERY;

-- 3. Aplicar log posterior al error usando STOPAT
RESTORE LOG QhatuPeru_Rescate
FROM DISK = 'C:\Backups\QhatuPeru_Log_2_afterError.trn'
WITH RECOVERY,
      STOPAT = '2025-11-15T14:35:15';
GO

```

2.8. Extraer los datos perdidos desde la base restaurada

```

--2.8. Extraer los datos perdidos desde la base restaurada
SELECT *
FROM QhatuPeru_Rescate.dbo.ARTICULO
WHERE IdArticulo = 5;

```

2.9. Reinsertar el dato recuperado en la base original

```

--2.9. Reinsertar el dato recuperado en la base original
INSERT INTO QhatuPeru.dbo.ARTICULO (IdArticulo, Nombre, Precio, Stock, Categoria)
SELECT IdArticulo, Nombre, Precio, Stock, Categoria
FROM QhatuPeru_Rescate.dbo.ARTICULO
WHERE IdArticulo = 5;
GO

```

2.10. (Opcional) Confirmar restauración

```

--2.10. (Opcional) Confirmar restauración
SELECT * FROM QhatuPeru.dbo.ARTICULO WHERE IdArticulo = 5;

```

2.11. Opcional: eliminar base temporal

```

--2.11. Opcional: eliminar base temporal
DROP DATABASE QhatuPeru_Rescate;

```

3. Justificación técnica de la solución

- **Modelo de recuperación FULL**
Es obligatorio para poder realizar backups de LOG y restauración punto-en-el-tiempo.
- **Backup FULL + Log antes del error**
Garantiza una cadena completa y consistente.
- **Backup Log después del error**
Permite identificar el rango del error y realizar un STOPAT correcto.
- **STOPAT**
Esta opción permite restaurar la base exactamente hasta antes del DELETE perjudicial.
- **Restauración en base alterna (QhatuPeru_Rescate)**
Evita interrumpir la base productiva y reduce riesgo de pérdida adicional.
- **Extracción selectiva de filas**
Es uno de los métodos más seguros y profesionales para recuperación granular (fila por fila).

4. Explicación de buenas prácticas utilizadas

- ✓ Siempre usar modelo FULL para PITR.
- ✓ Tomar backup de log antes de operaciones críticas, como cargas masivas o mantenimientos.
- ✓ Registrar hora exacta del error para un STOPAT preciso.
- ✓ Restaurar en una base alterna y no sobrescribir la producción.
- ✓ Extraer solo lo necesario, evitando restauraciones completas costosas.
- ✓ Verificar integridad de los backups con CHECKSUM.
- ✓ Mantener nombres lógicos y rutas organizadas para evitar confusiones en RESTORE.

Proyecto 14: Planes de mantenimiento – chequeos de integridad y tareas automáticas

1. Enunciado del ejercicio

Diseñar e implementar, mediante **scripts T-SQL**, un plan de mantenimiento semanal para la base de datos **QhatuPeru**, que incluya:

1. Chequeos de integridad (DBCC CHECKDB)

2. **Reconstrucción de índices con fragmentación mayor al 30%**
3. **Actualización de estadísticas**
4. **Limpieza automática de archivos de backup antiguos**, con período de retención configurable
5. (Opcional) Registro en tabla de auditoría de las tareas ejecutadas

No se utilizará el asistente gráfico; todo debe realizarse por script.

2. Script de la solución en T-SQL

Ajusta rutas y períodos de retención según tu entorno.

2.1. Crear tabla de auditoría del plan de mantenimiento

```
--pROYECTO 14
--2.1. Crear tabla de auditoría del plan de mantenimiento
CREATE TABLE dbo.MantenimientoAudit (
    Id INT IDENTITY PRIMARY KEY,
    Fecha DATETIME DEFAULT GETDATE(),
    Operacion VARCHAR(100),
    Detalle VARCHAR(MAX),
    Estado VARCHAR(20)
);
```

2.2. Script 1 — Chequeo de integridad semanal (DBCC CHECKDB)

```
--2.2. Script 1 - Chequeo de integridad semanal (DBCC CHECKDB)
BEGIN TRY
    INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
    VALUES ('CHECKDB', 'Iniciando chequeo de integridad', 'Inicio');

    DBCC CHECKDB (QhatuPeru) WITH NO_INFOMSGS, ALL_ERRORMSGS;

    INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
    VALUES ('CHECKDB', 'Chequeo completado sin errores', 'OK');
END TRY
BEGIN CATCH
    INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
    VALUES ('CHECKDB', ERROR_MESSAGE(), 'ERROR');
END CATCH;
```

2.3. Script 2 — Reconstrucción de índices fragmentados (>30%)

```
--2.3. Script 2 - Reconstrucción de índices fragmentados (>30%)
DECLARE @Table VARCHAR(300), @Index VARCHAR(300), @sql NVARCHAR(MAX);

DECLARE cur CURSOR FOR
SELECT OBJECT_NAME(ips.object_id), i.name
FROM sys.dm_db_index_physical_stats(DB_ID('QhatuPeru'), NULL, NULL, NULL, 'LIMITED') ips
JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 30
AND i.index_id > 0;

OPEN cur;
FETCH NEXT FROM cur INTO @Table, @Index;

WHILE @@FETCH_STATUS = 0
BEGIN
SET @sql = '
ALTER INDEX [' + @Index + '] ON [' + @Table + '] REBUILD WITH (ONLINE=ON);
';

BEGIN TRY
EXEC sp_executesql @sql;

INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
VALUES ('Rebuild Index', @Table + '.' + @Index, 'OK');
END TRY
BEGIN CATCH
INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
VALUES ('Rebuild Index', @Table + '.' + @Index + ' - ' + ERROR_MESSAGE(), 'ERROR');
END CATCH;

FETCH NEXT FROM cur INTO @Table, @Index;
END

CLOSE cur;
DEALLOCATE cur;
```

2.4. Script 3 — Actualización de estadísticas

```
--2.4. Script 3 - Actualización de estadísticas
BEGIN TRY
UPDATE STATISTICS QhatuPeru WITH FULLSCAN;

INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
VALUES ('Actualizar Estadísticas', 'Se ejecutó UPDATE STATISTICS con FULLSCAN', 'OK');
END TRY
BEGIN CATCH
INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
VALUES ('Actualizar Estadísticas', ERROR_MESSAGE(), 'ERROR');
END CATCH;
```

2.5. Script 4 — Limpieza de archivos antiguos de backup

Retención configurable: parámetro @DiasRetencion.

```
--2.5. Script 4 - Limpieza de archivos antiguos de backup
DECLARE @RutaBackups VARCHAR(300) = 'C:\Backups\';
DECLARE @DiasRetencion INT = 14;

DECLARE @cmd VARCHAR(8000);

SET @cmd = 'forfiles /p "" + @RutaBackups + "" /s /m *.bak /d -'
+ CAST(@DiasRetencion AS VARCHAR(10)) + ' /c "cmd /c del @path"';

BEGIN TRY
EXEC xp_cmdshell @cmd;

INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
VALUES ('Limpieza Backups', 'Backups mayores a ' + CAST(@DiasRetencion AS VARCHAR) + ' días eliminados', 'OK');
END TRY
BEGIN CATCH
INSERT INTO MantenimientoAudit (Operacion, Detalle, Estado)
VALUES ('Limpieza Backups', ERROR_MESSAGE(), 'ERROR');
END CATCH;
```

2.6. Crear un SQL Agent Job (demostración por script)

```
--2.6. Crear un SQL Agent Job (demostración por script)
USE msdb;
GO

EXEC sp_add_job @job_name = 'Plan_Mantenimiento_QhatuPeru';

EXEC sp_add_jobstep
    @job_name = 'Plan_Mantenimiento_QhatuPeru',
    @step_name = 'Tareas Mantenimiento',
    @subsystem = 'TSQL',
    @command = '
        -- CHECKDB
        DBCC CHECKDB (QhatuPeru) WITH NO_INFOMSGS;

        -- Reconstrucción de índices
        EXEC dbo.usp_RebuildIndices_QhatuPeru;

        -- Actualización de estadísticas
        UPDATE STATISTICS QhatuPeru WITH FULLSCAN;

        -- Limpieza de backups
        EXEC dbo.usp_LimpiezaBackups;
    ',
    @database_name = 'QhatuPeru';

EXEC sp_add_schedule
    @schedule_name = 'SemanalDomingo2AM',
    @freq_type = 8,           -- semanal
    @freq_interval = 1,      -- domingo
    @active_start_time = 020000; -- 02:00 am

EXEC sp_attach_schedule
    @job_name = 'Plan_Mantenimiento_QhatuPeru',
    @schedule_name = 'SemanalDomingo2AM';

EXEC sp_add_jobserver
    @job_name = 'Plan_Mantenimiento_QhatuPeru';
GO
```

3. Justificación técnica de la solución

- 1. DBCC CHECKDB**
Garantiza integridad física/lógica detectando corrupción temprana.
- 2. Rebuild de índices fragmentados (>30%)**
Mejora rendimiento eliminando fragmentación pesada.
Fragmentación leve se manejaría con reorganización (<30%).
- 3. Update Statistics con FULLSCAN**
Asegura que el optimizador tenga información actualizada para planes eficientes.
- 4. Limpieza automática de backups**
Evita llenado de disco.
Uso de **forfiles** permite una solución controlada y compatible con SQL Server.
- 5. Registro en tabla de auditoría**
Garantiza trazabilidad, transparencia de operación y facilita diagnóstico.

6. Ejecución semanal mediante SQL Agent

Automatiza todo el proceso sin intervención manual.

4. Explicación de las buenas prácticas utilizadas

- ✓ **Automatización total** mediante SQL Agent
- ✓ **Auditoría permanente** mediante tabla de registro
- ✓ **CHECKDB semanal** como mínimo recomendado por Microsoft
- ✓ **Rebuild solo cuando pasa 30%** para evitar reprocesos inútiles
- ✓ **Estadísticas FULLSCAN** ideal para cargas analíticas o tablas grandes
- ✓ **Limpieza controlada** evitando saturación del disco
- ✓ **Separación de pasos y procedimientos almacenados** para reuso y mantenimiento
- ✓ **Uso de TRY-CATCH** para captura de errores y registro automático

PROYECTO 15: Políticas de retención y compresión de backups (ahorro de espacio)

1. Enunciado del ejercicio

Implementar una estrategia de **retención y compresión de backups** para optimizar el uso de espacio en disco.

La política definida es:

- **FULL** → conservar **4 semanas** (28 días)
- **DIFERENCIAL (DIFF)** → conservar **7 días**
- **LOG** → conservar **48 horas**

Además:

- Todos los backups deben realizarse con **compresión (WITH COMPRESSION)**.
- Implementar un **script que elimine automáticamente** los archivos .bak/.trn fuera del periodo de retención, tomando como referencia `msdb.dbo.backupset`.

2. Script T-SQL de la solución

2.1. Backup FULL con compresión (semanal)

```
--Proyecto 15
--2.1. Backup FULL con compresión (semanal)
BACKUP DATABASE QhatuPeru
TO DISK = 'D:\Backups\Full\QhatuPeru_FULL_' + CONVERT(VARCHAR(10), GETDATE(), 112) + '.bak'
WITH COMPRESSION, FORMAT, INIT, STATS = 10;
```

2.2. Backup diferencial diario con compresión

```
--2.2. Backup diferencial diario con compresión
BACKUP DATABASE QhatuPeru
TO DISK = 'D:\Backups\Diff\QhatuPeru_DIFF_' + CONVERT(VARCHAR(10), GETDATE(), 112) + '.bak'
WITH DIFFERENTIAL, COMPRESSION, INIT, STATS = 10;
```

2.3. Backup de LOG cada 30 o 60 minutos (según el plan)

```
--2.3. Backup de LOG cada 30 o 60 minutos (según el plan)
BACKUP LOG QhatuPeru
TO DISK = 'D:\Backups\Log\QhatuPeru_LOG_'
+ REPLACE(CONVERT(VARCHAR(19), GETDATE(), 120), ':', '-') + '.trn'
WITH COMPRESSION, STATS = 5;
```

3. Script de eliminación automática según retención

3.1. Eliminar FULL > 28 días

```
-- 3.1. Eliminar FULL > 28 días
DECLARE @RetentionFull INT = 28;

DELETE b
FROM msdb.dbo.backupset b
WHERE b.database_name = 'QhatuPeru'
AND b.type = 'D'
AND b.backup_start_date < DATEADD(DAY, -@RetentionFull, GETDATE());
```

3.2. Eliminar DIFF > 7 días

```
--3.2. Eliminar DIFF > 7 días
DECLARE @RetentionDiff INT = 7;

DELETE b
FROM msdb.dbo.backupset b
WHERE b.database_name = 'QhatuPeru'
AND b.type = 'I'
AND b.backup_start_date < DATEADD(DAY, -@RetentionDiff, GETDATE());
```

3.3. Eliminar LOG > 48 horas

```
--3.3. Eliminar LOG > 48 horas
DECLARE @RetentionLog INT = 2; -- 2 días = 48 horas

DELETE b
FROM msdb.dbo.backupset b
WHERE b.database_name = 'QhatuPeru'
AND b.type = 'L'
AND b.backup_start_date < DATEADD(DAY, -@RetentionLog, GETDATE());
```

3.4. Eliminar físicamente los archivos expirados (PowerShell opcional)

(Suele ejecutarse desde un Job SQL)

```
--3.4. Eliminar físicamente los archivos expirados (PowerShell opcional)

(Suele ejecutarse desde un Job SQL)

EXEC xp_cmdshell 'forfiles /p "D:\Backups\Full" /s /m *.bak /d -28 /c "cmd /c del @file"';
EXEC xp_cmdshell 'forfiles /p "D:\Backups\Diff" /s /m *.bak /d -7 /c "cmd /c del @file"';
EXEC xp_cmdshell 'forfiles /p "D:\Backups\Log" /s /m *.trn /d -2 /c "cmd /c del @file";'
```

4. Justificación técnica de la solución aplicada

✓ Compresión de backups (WITH COMPRESSION)

- Reduce el tamaño entre **40% y 70%**.
- Acelera la escritura en disco.
- Reduce costos y tiempo de traslado de backups.

✓ Política de retención

- FULL semanales mantienen puntos de recuperación sólidos.
- DIFF diarios reducen tiempo de restauración entre días.
- LOG cada hora garantiza recuperación punto en el tiempo.
- Retención limitada evita saturación del disco.

✓ Uso de **msdb.backupset** para limpieza

- Garantiza eliminar únicamente backups ya catalogados.
- Evita borrar archivos en uso o necesarios.
- Proporciona auditoría natural por la metadata del servidor.

✓ xp_cmdshell + forfiles

- Metodo estándar para limpiar archivos físicos.
- Permite mantener discos ordenados sin intervención manual.

5. Buenas prácticas aplicadas en este proyecto

Compresión siempre activada

Ahorra espacio y mejora rendimiento.

Rutas separadas por tipo de backup

Organización: Full/, Diff/, Log/.

Convenciones de nombres con fecha

Facilita auditoría y restauración.

Política de retención documentada

Respalda cumplimiento de normas y evita crecimiento descontrolado.

Control basado en msdb

Evita borrar archivos esenciales y garantiza integridad del historial.

Automatización vía SQL Agent Jobs

Reduce errores humanos y garantiza ejecución puntual.